

Benchmark CPU vs GPU sur opérations vectorielles

Rapport technique — Systèmes Répartis / Calcul Parallèle

Binôme : Hajar En-Najdy & Mohamed Akrame Kraichy

Environnement de mesure : Google Colab — GPU Tesla T4 (15,64 Go) — PyTorch 2.11.0 + CUDA 12.8

Type de données : vecteurs float32 aléatoires | **Tailles :** 1 000 à 100 000 000 éléments

1. Objectif

Ce TP compare les performances du **CPU** et du **GPU** sur trois opérations vectorielles fondamentales — la **somme**, le **produit scalaire** et la **norme** — pour des vecteurs de taille croissante. L'objectif est de déterminer à partir de quelle taille le GPU devient avantageux (le **point de bascule**), de quantifier l'accélération obtenue, et de comprendre pourquoi le GPU n'est pas systématiquement plus rapide.

2. Méthodologie

2.1 Opérations mesurées

Opération	Formule	Vecteurs lus
Somme	$S = \sum a_i$	1
Produit scalaire	$a \cdot b = \sum a_i b_i$	2
Norme	$\ a\ = \sqrt{(\sum a_i^2)}$	1

2.2 Protocole de mesure fiable

Trois précautions garantissent la validité des mesures :

- Warm-up** : plusieurs exécutions « à blanc » non mesurées, pour écarter le surcoût de la première exécution (allocation mémoire, initialisations internes de PyTorch).
- Synchronisation GPU** : `torch.cuda.synchronize()` force l'attente de la fin réelle du calcul. Sans elle, on ne mesurerait que le temps de *lancer* l'ordre, le GPU travaillant de façon asynchrone — d'où des temps faussement minuscules.
- Médiane** sur 10 répétitions : robuste face à une mesure ponctuellement perturbée par le système, contrairement à la moyenne.

Note importante : les données sont créées directement dans la mémoire de l'appareil cible (CPU ou GPU) *avant* le démarrage du chronomètre. On mesure ainsi le **temps de calcul pur**, sans inclure le coût de transfert des données vers le GPU.

3. Résultats

Temps médian d'exécution (en millisecondes) et accélération GPU (speedup = temps CPU / temps GPU). Un speedup > 1 signifie que le GPU est plus rapide ; < 1, que le CPU l'emporte.

Opération	Taille	CPU (ms)	GPU (ms)	Speedup	Gagnant
Somme	1 000	0.0036	0.0232	0.16	CPU
	10 000	0.0040	0.0247	0.16	CPU
	100 000	0.0076	0.0382	0.20	CPU
	1 000 000	0.1762	0.0372	4.74	GPU
	10 000 000	2.3382	0.1763	13.26	GPU
	100 000 000	33.4152	1.4977	22.31	GPU
Produit scalaire	1 000	0.0027	0.0259	0.10	CPU
	10 000	0.0036	0.0270	0.13	CPU
	100 000	0.0097	0.0266	0.36	CPU
	1 000 000	0.3306	0.0550	6.01	GPU
	10 000 000	5.0440	0.3241	15.56	GPU
	100 000 000	60.3430	3.1425	19.20	GPU
Norme	1 000	0.0057	0.0291	0.19	CPU
	10 000	0.0074	0.0332	0.22	CPU
	100 000	0.0258	0.0424	0.61	CPU
	1 000 000	0.2033	0.0385	5.28	GPU
	10 000 000	3.1905	0.1760	18.13	GPU
	100 000 000	33.9037	1.5013	22.58	GPU

3.1 Temps d'exécution en fonction de la taille

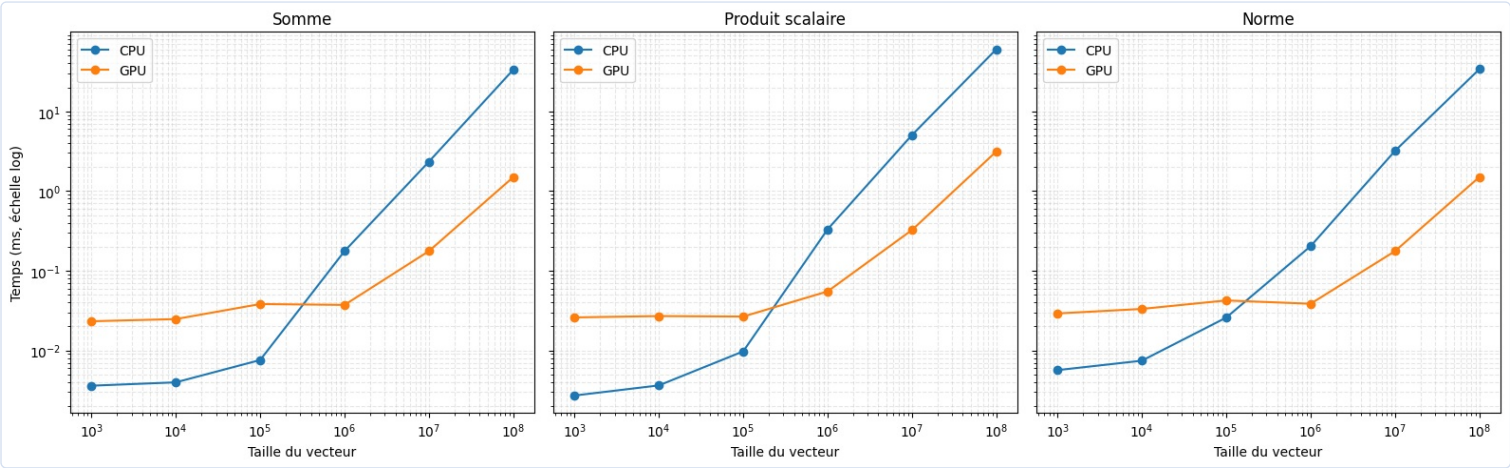


Figure 1 — Temps (échelle log-log). Le croisement des courbes CPU et GPU marque le point de bascule.

3.2 Accélération (speedup) en fonction de la taille

Accélération GPU selon la taille

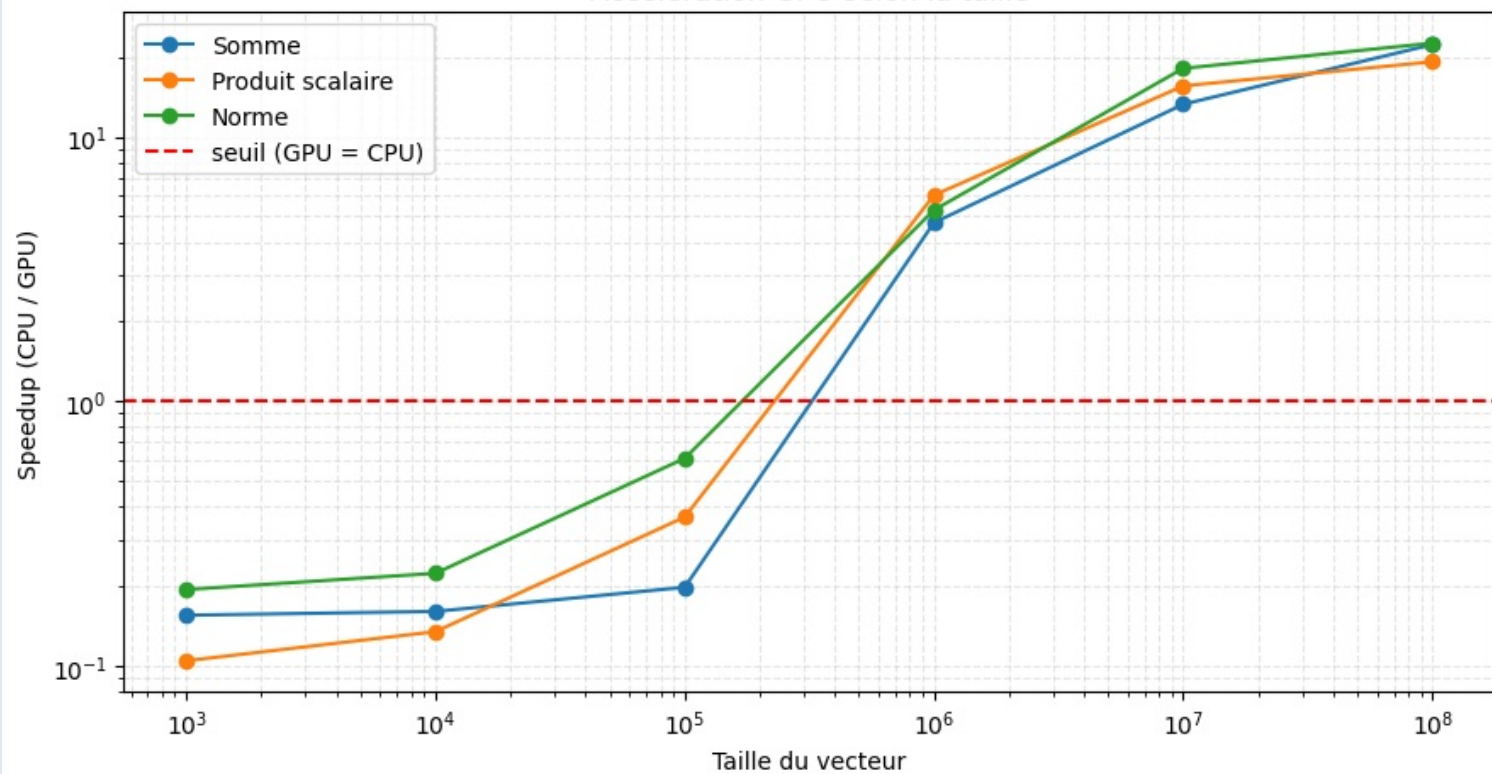


Figure 2 — Speedup GPU. Au-dessus de la ligne rouge ($y=1$), le GPU est plus rapide.

4. Analyse

4.1 Petits vecteurs (1K - 100K) : le CPU domine

Pour les petites tailles, le GPU est **plus lent** que le CPU (speedup entre 0,10 et 0,61). Le temps GPU reste quasi constant autour de 0,02-0,04 ms quelle que soit la taille : ce plancher correspond au **coût fixe de lancement du kernel** (l'ordre envoyé au GPU). Comme le calcul lui-même est minuscule à ces tailles, ce coût fixe domine et le GPU ne peut pas exprimer son parallélisme.

4.2 Point de bascule : autour de 1 000 000 d'éléments

Sur les trois opérations, le GPU passe devant le CPU **entre 100 000 et 1 000 000 d'éléments**. Dès 1M, le speedup dépasse nettement 1 (de 4,7 à 6,0 selon l'opération). C'est le seuil à partir duquel la quantité de calcul devient suffisante pour que le parallélisme massif du GPU compense son coût de lancement.

4.3 Grands vecteurs (10M - 100M) : le GPU domine largement

À 100 millions d'éléments, le GPU est **19 à 23 fois plus rapide** que le CPU. Le temps CPU croît linéairement avec la taille (il traite les éléments par paquets), tandis que le GPU répartit le travail sur ses milliers de cœurs en parallèle. C'est le régime où le GPU est pleinement justifié.

4.4 Différences entre opérations

Le **produit scalaire** est l'opération la plus coûteuse sur CPU (60,3 ms à 100M, contre ~33 ms pour la somme et la norme), car il lit **deux** vecteurs et effectue une multiplication avant la réduction. Le CPU souffre donc davantage de ce surcroît de données, ce qui rend l'apport du GPU particulièrement visible sur cette opération aux tailles intermédiaires.

5. Conclusion

Ce benchmark confirme que **le GPU n'est pas universellement plus rapide** : son avantage dépend de la taille du problème. En dessous de $\sim 10^5$ éléments, le coût de lancement du kernel rend le CPU préférable. Au-delà de $\sim 10^6$, le parallélisme du GPU prend le dessus, avec une accélération atteignant **$\sim 23\times$** à 100 millions d'éléments sur une Tesla T4. Le point de bascule observé se situe autour de 10^6 éléments pour les trois opérations testées.

À retenir : le choix CPU/GPU est un arbitrage qui dépend de la taille des données. Pour de petits calculs fréquents, le CPU reste pertinent ; pour de gros volumes, le GPU offre un gain d'un ordre de grandeur. Et toute mesure GPU fiable exige **synchronisation, warm-up** et **médiane**.